Guía del proyecto

Calculadora en Python

Repositorio: AlejandraSaldana/actividad-nlp-2

Clase 8 - Continuación de la calculadora a las pruebas automáticas.

Esta guía explica cómo ejecutar el proyecto, comprobar sus cambios y reunir la evidencia de entrega. Los resultados finales deben corresponder al commit que se entregue.

1. Preparar y ejecutar

Utiliza Python 3.12 o posterior. Abre una terminal en la carpeta del repositorio y crea un entorno virtual:

```
python -m venv .venv
```

Windows, Símbolo del sistema:

```
.venv\Scripts\activate.bat
```

Windows, PowerShell:

```
.venv\Scripts\Activate.ps1
```

macOS o Linux:

```
source .venv/bin/activate
```

Instala las dependencias y abre el menú:

```
python -m pip install -r requirements.txt
python calculadora.py
```

No guardes el entorno virtual, credenciales, cachés ni reportes temporales en los commits. El PDF de esta guía sí es un documento del proyecto.

2. Calculadora y API

La clase Calculadora guarda la operación y los números. Cada operación tiene su método; resultado() valida las entradas y elige el método correspondiente.

Operación	Ejemplo	Resultado
suma	2, 3	5
resta	3, 8	-5
multiplicacion	2, 3, 0.5	3
division	9, 2	4.5
promedio	8, 9, 10	9
potencia	2, 3	8
mayor	4, -2, 8	8
menor	4, -2, 8	-2

En el menú, resta, división y potencia solicitan dos números en orden. Las demás operaciones solicitan una cantidad y una lista. La opción 9 sale; después de calcular, responde s para continuar o n para terminar. La división entre cero y los datos incorrectos deben mostrar mensajes comprensibles.

La API reutiliza la misma clase. Iníciala con:

```
python -m uvicorn api:aplicacion --host 127.0.0.1 --port 8000
```

El nombre exportado es aplicacion. GET /estado confirma que el servicio funciona. POST /calcular recibe este cuerpo JSON:

```
{"operacion": "promedio", "numeros": [8, 9, 10]}
```

Debe devolver un resultado numérico de 9. No uses /docs para comprobar el servicio: la documentación interactiva está desactivada. Para el escaneo de ZAP en GitHub, el workflow debe iniciar la API y comprobar /estado antes de escanearla.

3. Pruebas y cobertura

Una prueba unitaria comprueba una operación con entradas conocidas y compara el resultado esperado. Una prueba de integración conecta partes: respuestas del usuario y menú, ejecución del programa, o petición HTTP y clase Calculadora.

```
python -m pytest pruebas/test_calculadora.py -v
python -m pytest pruebas/test_integracion.py -v
python -m pytest pruebas/test_guia.py -v
python -m pytest --cov --cov-report=term-missing --cov-report=xml
```

La entrega exige al menos 90 % de cobertura combinada de líneas y ramas, incluyendo calculadora.py, api.py y las herramientas Python incorporadas. Las pruebas no deben contarse como aplicación. Revisa la lista de archivos del informe: si una herramienta no aparece, el porcentaje no demuestra su cobertura.

Una prueba propia ya incluida es promedio_de_calificaciones: calcula el promedio de 70, 85.5, 92 y 100 y espera 86.875. Comprueba decimales y datos distintos de los ejemplos. pytest.approx permite comparar números de punto flotante con una tolerancia adecuada.

Las herramientas de consulta y reporte también necesitan pruebas: errores de red, análisis fallido, correspondencia del commit, datos ausentes y Quality Gate no aprobado. Esta guía no sustituye esas pruebas.

Antes de publicar:

```
python -m ruff check .
python -m ruff format --check .
git status
```

Si falla el formato, aplica python -m ruff format sobre los archivos que te corresponden y repite la verificación. No reduzcas el umbral de cobertura para obtener una ejecución verde.

4. GitHub Actions y SonarQube

CI se activa con push a main, pull_request y ejecución manual. needs establece qué trabajos deben terminar antes de iniciar otro. Así, el análisis espera a que exista coverage.xml y hayan pasado los controles previos.

Control	Responsabilidad
revision	Ruff: reglas y formato.
unitarias	Pruebas de operaciones y de reportes requeridas.
integracion	Menú y API; espera a unitarias.
cobertura	Todas las pruebas; espera a unitarias e integracion.
secretos / seguridad	Gitleaks busca credenciales; Bandit revisa Python.
dependencias / obsoletas	Vulnerabilidades y versiones nuevas; obsoletas es informativo.
zap	Escanea la API tras integración y controles de seguridad.
build-and-analyze	SonarQube; espera cobertura, revisión y seguridad.
reporte	Descarga resultados y comprueba el Quality Gate.
resumen	Muestra los estados, incluso si algo falla.

El identificador actual del análisis es build-and-analyze; la consigna y la referencia lo llaman sonarqube. El análisis y su reporte se ejecutan en main y se omiten en los PR. CI en verde requiere que los controles obligatorios aprueben; revisa la ejecución completa.

Servidor: https://sonarqube.lynvera.ai

Clave configurada en sonar-project.properties: team02_actividad02. Confirma que sea la asignada por el docente. Guarda SONAR_TOKEN en Settings, Secrets and variables, Actions. Debe ser User Token con Browse y Execute Analysis; nunca lo incluyas en un archivo ni lo compartas en una captura.

El escáner debe importar coverage.xml. El trabajo de reporte necesita el archivo .scannerwork/report-task.txt del análisis. Los trabajos tienen carpetas independientes: ese archivo debe transferirse como artefacto o consumirse en el mismo trabajo.

5. Reportes y descarga

En Actions, abre una ejecución del commit de entrega y busca Artifacts al final de la página. Descarga sonarqube-report-pdf-json-md y comprueba que incluya sonarqube.pdf, sonarqube.json y sonarqube.md.

El generador de reportes guarda sus documentos en reports/sonarqube/. La ruta de publicación del workflow debe coincidir con esa carpeta. Si no aparece el artefacto, revisa los registros de reporte y de los trabajos que lo preceden.

El PDF de SonarQube debe identificar el Quality Gate, cobertura general y nueva, issues totales y nuevos, proyecto, fecha y commit. Compara el commit del PDF con el de la ejecución de GitHub.

El código nuevo depende de la línea base configurada en SonarQube; no necesariamente equivale al último commit. N/D significa que no hay una medición disponible y no debe interpretarse como cero. Quality Gate aprobado y 0 issues son comprobaciones distintas: revisa ambas.

Generar esta guía

GUIA_ALUMNOS.md es la fuente editable. Desde la raíz del repositorio:

```
python -m herramientas.guia_pdf
```

El resultado es material/GUIA_ALUMNOS.pdf. El workflow independiente Guía del proyecto genera el mismo documento y publica guia-para-alumnos-pdf. Se ejecuta al cambiar estos documentos en PR/main y también de forma manual. Una guía publicada no demuestra que CI haya aprobado.

Resolver un fallo

Abre el primer trabajo fallido, lee la causa, corrige en la misma rama y revisa la ejecución nueva. Por ejemplo, Attribute app not found indica que el comando de arranque no coincide con api:aplicacion. Una conexión rechazada de ZAP requiere comprobar primero que la API arrancó.

6. Evidencia y entrega

Comprueba estos puntos en la ejecución final de main:

• CI aprobado, con SonarQube y reporte ejecutados.

• Quality Gate aprobado y 0 issues en el escaneo.

• Cobertura mínima de 90 % con todos los archivos requeridos incluidos.

• Reporte PDF, JSON y Markdown descargables y del commit correcto.

• Guía actualizada y su PDF descargable.

Presenta:

• Enlace al repositorio público y al pull request.

• Enlace a la ejecución aprobada de main.

• Archivo calculadora.py y reporte sonarqube.pdf.

• Guía del proyecto actualizada y su PDF.

• Explicación breve de una prueba propia, un fallo corregido y los resultados de cobertura e issues.

Para explicar el trabajo, usa la prueba de promedio de calificaciones, describe un fallo que hayas corregido realmente y toma cobertura e issues de la evidencia final. No declares 0 issues ni cobertura completa si aún no se han comprobado. Debes poder explicar qué diferencia una prueba unitaria de una de integración, por qué se usa needs y dónde están los artefactos.

Referencias

Repositorio del equipo:

https://github.com/AlejandraSaldana/actividad-nlp-2

Proyecto docente de referencia:

https://github.com/felipecastillotecmx/calculadora

Guía docente original: material/GUIA_ALUMNOS.pdf dentro del proyecto de referencia. Este documento adapta sus objetivos a los nombres de archivos, comandos y clave configurada del equipo. La consigna de clase es la referencia para evaluar la entrega.